

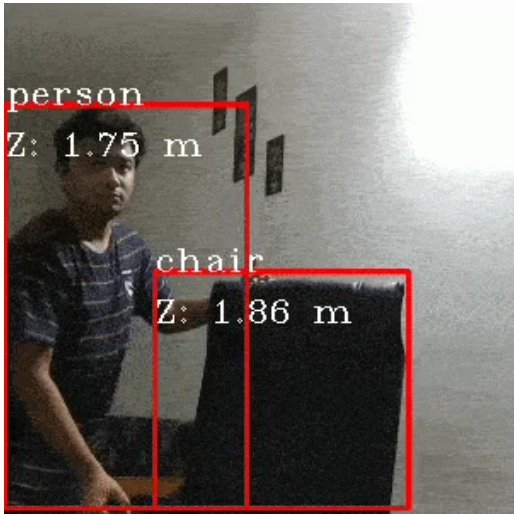
Introduction to Epipolar Geometry and Stereo Vision

 Kaustubh Sadekar (https://learnopencv.com/author/kaustubh-sadekar/)

DECEMBER 28, 2020

[3D Computer Vision \(https://learnopencv.com/category/3d-computer-vision/\)](https://learnopencv.com/category/3d-computer-vision/) [Classical Computer Vision \(https://learnopencv.com/category/classical-computer-vision/\)](https://learnopencv.com/category/classical-computer-vision/) [Edge Devices \(https://learnopencv.com/category/edge-devices/\)](https://learnopencv.com/category/edge-devices/) [OAK \(https://learnopencv.com/category/oak/\)](https://learnopencv.com/category/oak/)

Have you ever wondered why you can experience that wonderful 3D effect when you watch a movie with those special 3D glasses? Or why is it difficult to catch a cricket ball with your one eye closed? It all relates to stereoscopic vision, which is our ability to perceive depth using both the eyes. This post uses OpenCV and stereo vision to give this power of perceiving depth to a computer. The code is provided in Python and C++.



GIF showing object detection along with distance

The cool part about the above GIF is that besides detecting different objects, the computer is also able to tell how far they are. Which means it can perceive depth! For this video, the stereo camera setup of OAK-D (OpenCV AI Kit- Depth) (https://www.kickstarter.com/projects/opencv/opencv-ai-kit) was used to help the computer perceive depth.

What is a stereo camera setup? How do we use it to provide a sense of depth to a computer? Does it have anything to do with stereoscopic vision?

This post will try to answer these questions by understanding fundamental concepts related to epipolar geometry and stereo vision.

Most of the post’s theoretical explanations are inspired by the book: **Multiple View Geometry in Computer Vision by Richard Hartley and Andrew Zisserman**. It is a very famous and standard textbook for understanding various fundamental concepts of computer vision.

This post is the first part of the **Introduction to Spatial AI series**. It provides a detailed introduction to various fundamental concepts and creates a strong foundation for the subsequent parts of the series.

Great! So let’s get started and help our computer to perceive depth!

FREE



Python Bootcamp

Enroll Now!

 OpenCV University

(https://opencv.org/university/python-for-beginners/?utm_source=locv&utm_medium=midblog&utm_campaign=introduction-to-epipolar-geometry-and-stereo-vision)

FREE official



OpenCV Bootcamp

Enroll Now!

 OpenCV University

(https://opencv.org/university/free-opencv-course/?utm_source=locv&utm_medium=midblog&utm_campaign=introduction-to-epipolar-geometry-and-stereo-vision)

FREE



TensorFlow Bootcamp

Enroll Now!

 OpenCV University

(https://opencv.org/university/free-tensorflow-keras-course/?utm_source=locv&utm_medium=midblog&utm_campaign=introduction-to-epipolar-geometry-and-stereo-vision)

Do we need more than one image to calculate depth?

When we capture (project) a 3D object in an image, we are projecting it from a 3D space to a **2D (planar) projective space**. This is called **the Planar Projection**. The problem is that we lose the depth information due to this planar projection.

So how do we recover the depth? Can we calculate back the depth of a scene using a single image? Let’s try a simple example.

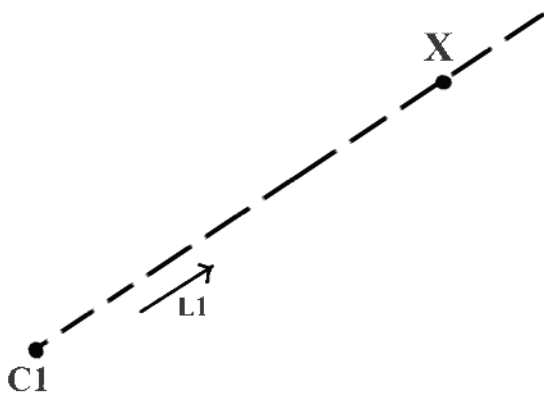


Figure 1 – Locating a 3D point (X), at an unknown depth, with a single known 3D point (C1) and direction vector (L1).

In figure 1, C1 and X are points in 3D space, and the unit vector L1 gives the direction of the ray from C1 through X. Now, can we find X if we know the values of point C1 and direction vector L1? Mathematically it simply means to solve for X in the equation

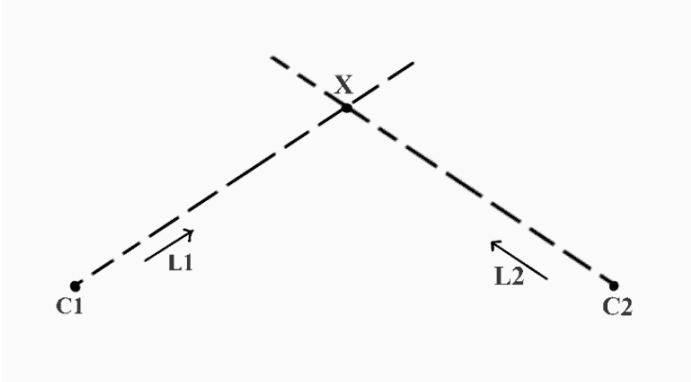
$$X = C1 + L1 \cdot d$$

$$A = C_1 + \kappa(L_1)$$



[Click here to download the source code to this post](#)

Now, as the value of k is not known, we cannot find a unique value of X.

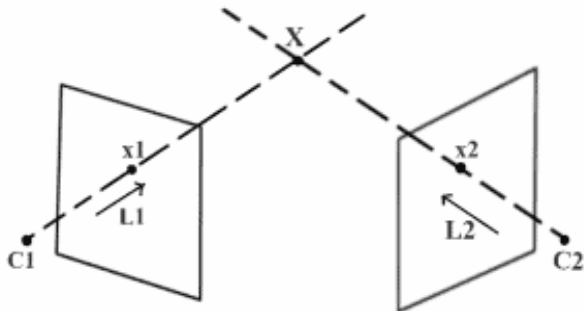


[./wp-content/uploads/2020/09/Point-recovery-in-3D-unique-solution-edited-1.png](#)

Figure 2 – Locating a 3D point (X), at an unknown depth, with two known 3D points (C1 and C2) and direction vectors (L1 and L2) – Triangulation.

In figure 2, we have an additional point C2, and L2 is the direction vector of the ray from C2 through X. Now can we find a unique value for X if C2 and L2 are also known to us?

Yes! Because the rays originating from C1 and C2 clearly intersect at a unique point, point X itself. This is called **triangulation**. We say we triangulated point X.



[./wp-content/uploads/2020/09/Triangulation-of-single-3D-point-in-two-view-geometry-edited-1.png](#)

Figure 3 – Extending the triangulation concept to explain how a 3D point (X) captured in two images can be calculated if the camera positions (C1 and C2) and pixel coordinates (x1 and x2) are known.

Figure 3 shows how triangulation can be used to calculate the depth of a point (X) when captured(projected) in two different views(images). In this figure, C1 and C2 are known 3D positions of the left and right cameras, respectively. x1 is the image of the 3D point X captured by the left camera, and x2 is the image of X captured by the right camera. x1 and x2 are called corresponding points because they are the projection of the same 3D point. We use x1 and C1 to find L1 and x2 and C2 to find L2. Hence we can use triangulation to find X just like we did for figure 2.

From the above example, we learned that to triangulate a 3D point using two images capturing it from different views, the key requirements are:

1. **Position of the cameras** – C1 and C2.
2. **Point correspondence** – x1 and x2.

Great! It is now clear that **we need more than one image** to find depth.

Hey! but this was just a single 3D point that we tried to calculate. How do we calculate a 3D structure of a real-world scene by capturing it from two different views? The obvious answer is by repeating the above process for all the 3D points captured in both the views. Let’s have a closer look at the practical challenges in doing this. Time for the reality check!

Practical and theoretical understanding of the two-view geometry



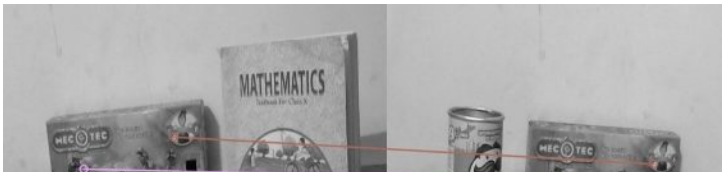
[./wp-content/uploads/2020/09/real-world-scene-captured-from-two-different-view-points-1024x378.jpg](#)

Figure 4 – Real world scene captured from two different view points.

Figure 4 shows two images capturing a real-world scene from different viewpoints. To calculate the 3D structure, we try to find the two key requirements mentioned before:

1. **The position of cameras** in the real world coordinate system (C1 and C2). We simplify this problem by calculating the 3D points by assuming one of the camera positions (C1 or C2) as the origin. We find it by **calibrating** the two view system using a known calibration pattern. This process is called stereo calibration.
2. **The point correspondence** (x1 and x2) for each 3D point (X) in the scene to be calculated. We will discuss various improvements for calculating point correspondence and finally understand how epipolar geometry can help us to simplify the problem.

Note that the stereo camera calibration is useful only when the images are captured by a pair of cameras rigidly fixed with respect to each other. If a single camera captures the images from two different angles, then we can find depth only to a scale. The absolute depth is unknown unless we have some special geometric information about the captured scene that can be used to find the actual scale.





[Click here to download the source code to this post](#)

[./wp-content/uploads/2020/09/Hand-picked-feature-matches-edited.jpg](#)

Figure 5 – Hand picked feature matches.

Figure 5 shows different matched points that were manually marked. It is easy for us to identify the corresponding points, but how do we make a computer do that?

One method which people regularly use in the computer vision community is called **feature matching**. Following figure 6 shows matched features between the left and right images using ORB feature descriptors. This is one method to find point correspondence (matches).

[./wp-content/uploads/2020/09/Feature-Matching-output.jpg](#)

Figure 6 – Results of feature detection algorithm.

However, we observe that the ratio of the number of pixels with known point correspondence to the total number of pixels is minimal. This means we will have a very **sparsely reconstructed 3D scene**. For dense reconstruction, we need to obtain point correspondence for the maximum number of pixels possible.

[./wp-content/uploads/2020/09/Multiple-matched-points-using-](#)

[template-matching-edited.jpg](#)

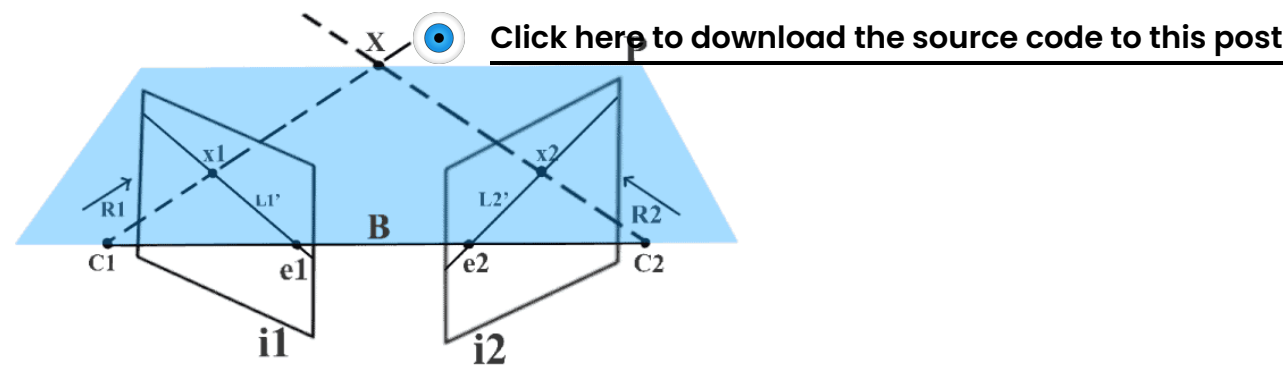
Figure 7 – Multiple matched points using template matching.

A simplified way to find the point correspondences is to find pixels with similar neighboring pixel information. In figure 7, we observe that using this method of matching pixels with similar neighboring information results in a single-pixel from one image having multiple matches in the other image. We find it challenging to write an algorithm to determine the true match.

Is there a way to reduce our search space? Some theorem which we can use to eliminate all the extra false matches that lead to inaccurate correspondence? We make use of epipolar geometry here.

All this explanation and build-up was to introduce the concept of **epipolar geometry**. Now we will understand the importance of epipolar geometry in reducing search space for point correspondence.

Epipolar geometry and its use in point correspondence



[Click here to download the source code to this post](#)

Figure 8 – Image explaining epipolar geometry.

In figure 8, we assume a similar setup to figure 3. A 3D point X is captured at x_1 and x_2 by cameras at C_1 and C_2 , respectively. As x_1 is the projection of X , if we try to extend a ray R_1 from C_1 that passes through x_1 , it should also pass through X . This ray R_1 is captured as line L_2 , and X is captured as x_2 in the image i_2 . **As X lies on R_1 , x_2 should lie on L_2 .** This way, the possible location of x_2 is constrained to a single line, and hence we can say that the **search space** for a pixel in image i_2 , corresponding to pixel x_1 , is **reduced to a single line L_2** . We use epipolar geometry to find L_2 .

Time to define some technical terms now! Along with X , we can also project the camera centers in the respective opposite images. e_2 is the projection of camera center C_1 in image i_2 , and e_1 is the projection of camera center C_2 in image i_1 . The technical term for **e_1 and e_2** is **epipole**. Hence in a two-view geometry setup, an epipole is the image of the camera center of one view in the other view.

The line joining the two camera centers is called a **baseline**. Hence epipole can also be defined as the intersection of **baseline** with the image plane.

Figure 8 shows that using R_1 and baseline, we can define a plane P . This plane also contains X , C_1 , x_1 , x_2 , and C_2 . We call this plane the **epipolar plane**. Furthermore, the line obtained from the intersection of the epipolar plane and the image plane is called **the epipolar line**. Hence in our example, L_2 is an epipolar line. For different values of X , we will have different epipolar planes and hence different epipolar lines. However, all the epipolar planes intersect at baseline, and all the epipolar lines intersect at epipole. All this together forms the epipolar geometry.

Revisiting figure 8 with all the technical terms we have learned till now.

We have epipolar plane P created using baseline B and ray R_1 . e_1 and e_2 are epipoles, and L_2 is the epipolar line. Based on the epipolar geometry of the given figure, search space for pixel in image i_2 corresponding to pixel x_1 is constrained to a single 2D line which is the epipolar line L_2 . This is called the epipolar constraint.

Is there a way to represent the entire epipolar geometry by a single matrix? Furthermore, can we calculate this matrix using just the two captured images? The good news is that there is such a matrix, and it is called **the Fundamental matrix**.

In the next two sections, we first understand what we mean by projective geometry and homogeneous representation and then try to derive the Fundamental matrix expression. Finally, we calculate the epipolar lines and represent the epipolar constraint by using the fundamental matrix.

Understanding projective geometry and homogeneous representation

How do we represent a line in a 2D plane? Equation of a line in a 2D plane is $ax + by + c = 0$. With different values of a , b , and c , we get different lines in a 2D plane. Hence a vector (a, b, c) can be used to represent a line.

Suppose we have line l_1 defined as $2x + 3y + 7 = 0$ and line l_2 as $4x + 6y + 14 = 0$. Based on our above discussion, l_1 can be represented by the vector $(2, 3, 7)$ and l_2 by the vector $(4, 6, 14)$. We can easily say that l_1 and l_2 essentially represent the same line and that the vector $(4, 6, 14)$ is basically the scaled version of the vector $(2, 3, 7)$, scaled by a factor of 2.

Hence any two vectors (a, b, c) and $k(a, b, c)$, where k is a non-zero scaling constant, represent the same line. Such **equivalent** vectors, which are related by just a scaling constant, form a class of **homogeneous vectors**. The vector (a, b, c) is the **homogeneous representation** of its respective equivalent vector class.

The set of all equivalent classes, represented by (a, b, c) , for all possible real values of a , b , and c other than $a=b=c=0$, forms the **projective space**. We use the homogeneous representation of homogeneous coordinates to define elements like points, lines, planes, etc., in projective space. We use the rules of **projective geometry** to perform any transformations on these elements in the projective space.



FREE

Python Bootcamp

Enroll Now!



FREE

OpenCV Bootcamp

Enroll Now!



FREE

TensorFlow Bootcamp

Enroll Now!

<https://opencv.org/university/python-for-beginners/>

<https://opencv.org/university/free-opencv-course/>

<https://opencv.org/university/free-tensorflow-keras-course/>

https://opencv.org/university/python-for-beginners/?utm_source=locv&utm_medium=midblog&utm_campaign=introduction-to-epipolar-geometry-and-stereo-vision

https://opencv.org/university/free-opencv-course/?utm_source=locv&utm_medium=midblog&utm_campaign=introduction-to-epipolar-geometry-and-stereo-vision

https://opencv.org/university/free-tensorflow-keras-course/?utm_source=locv&utm_medium=midblog&utm_campaign=introduction-to-epipolar-geometry-and-stereo-vision

Fundamental matrix derivation

In figure 3, Assume that we know the camera projection matrices for both the cameras, say P_1 for the camera at C_1 and P_2 for the camera at C_2 .

What is a projection matrix? The camera's projection matrix defines the relation between the 3D world coordinates and their corresponding pixel coordinates when captured by the camera. To know more about the camera projection matrix, read [this post on camera calibration](https://learnopencv.com/camera-calibration-using-opencv/) (<https://learnopencv.com/camera-calibration-using-opencv/>).

Just like P_1 projects 3D world coordinates to image coordinates, we define P_1^{-1} , the pseudo inverse of P_1 , such that we can define the ray R_1 from C_1 passing through x_1 and X as:



[Click here to download the source code to this post](#)

k is a scaling parameter as we do not know the actual distance of X from C_1 . We need to find the epipolar line L_{n2} to reduce the search space for a pixel in i_2 corresponding to pixel x_1 in i_1 as we know that L_{n2} is the image of ray R_1 captured in i_2 . Hence to calculate L_{n2} , we first find two points on ray R_1 , project them in image i_2 using P_2 and use the projected images of the two points to find L_{n2} .

The first point that we can consider on R_1 is C_1 , as the ray starts from this point. The second point can be calculated by keeping $k=0$. Hence we get the points as C_1 and $(P_1^{-1})(x_1)$.

Using the projection matrix P_2 we get the image coordinates of these points in the image i_2 as $P_2 \cdot C_1$ and $P_2 \cdot P_1^{-1} \cdot x_1$ respectively. We also observe that $P_2 \cdot C_1$ is basically the epipole e_2 in image i_2 .

A line can be defined in projective geometry using two points p_1 and p_2 by simply finding their cross product $p_1 \times p_2$. Hence

In projective geometry, if a point x lies on a line L , we can write it in the form of the equation

Hence, as x_2 lies on the epipolar line L_{n2} , we get

By replacing the value of L_{n2} from the above equation, we get the equation:

This is a necessary condition for the two points x_1 and x_2 to be corresponding points, and it is also a form of epipolar constraint. Thus F represents the overall epipolar geometry of the two-view system.

What else is so special about this equation? It can be used to find the epipolar lines!

Using Fundamental matrix to find epipolar lines

As x_1 and x_2 are corresponding points in the equation, if we can find correspondence for some points, using feature matching methods like ORB or SIFT, we can use them to solve the above equation for F .

The ***findFundamentalMat()*** method of OpenCV provides implementations of various algorithms, like 7-Point Algorithm, 8-Point Algorithm, RANSAC algorithm, and LMedS Algorithm, to calculate Fundamental matrix using matched feature points.

Once F is known, we can find the epipolar line L_{n2} using the formula

If we know L_{n2} , we can restrict our search for pixel x_2 corresponding to pixel x_1 using the epipolar constraint.

A special case of two-view vision – parallel imaging planes

We have been trying to solve the correspondence problem. We started by using feature matching, but we observed that it leads to a sparse 3D

structure, as the point correspondence for a tiny fraction of the total pixels is known. Then we saw how we could use a template-based search for pixel correspondence. We learned how epipolar geometry could be used to reduce the search space for point correspondence to a single line – the epipolar line.

[Click here to download the source code to this post](#)

Can we simplify this process of finding dense point correspondences even further?

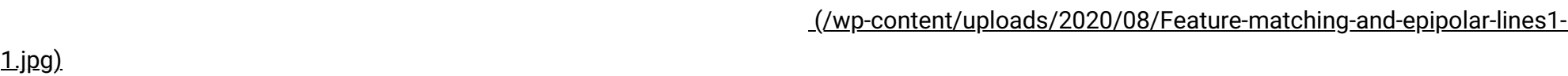
1.jpg).

Figure 9. Upper pair of images showing results for feature matching and lower pair of images showing a points in one image (left) and the corresponding points lying on respective epipolar lines in the second image (right).

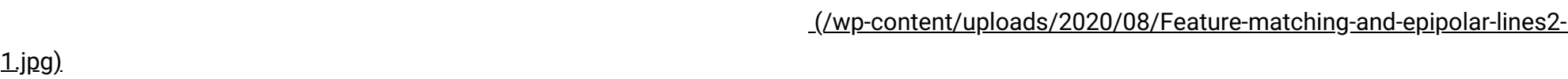
1.jpg).

Figure 10. A Special case of two view geometry. Upper pair of images showing results for feature matching and lower pair of images showing a points in one image (left) and the corresponding points lying on respective epipolar lines in the second image (right).
[Source – 2005 Stereo Dataset \(https://vision.middlebury.edu/stereo/data/scenes2005/\)](https://vision.middlebury.edu/stereo/data/scenes2005/).

Figure 9 and Figure 10 show the feature matching results and epipolar line constraint for two different pairs of images. What is the most significant difference between the two figures in terms of feature matching and the epipolar lines?

Yes! You got it right! The matched feature points have equal vertical coordinates in Figure 10. All the corresponding points have equal vertical coordinates. All the epipolar lines in Figure 10 have to be parallel and have the same vertical coordinate as the respective point in the left image. Well, what is so great about that?

Exactly! Unlike the case of figure 9, there is no need to calculate each epipolar line explicitly. If the pixel in the left image is at (x_1, y_1) , the equation of the respective epipolar line in the second image is $y=y_1$.

We search for each pixel in the left image for its corresponding pixel in the same row of the right image. This is a special case of two-view geometry where the imaging planes are parallel. Hence, the epipoles (image of one camera captured by the other camera) form at infinity. Based on our understanding of epipolar geometry, epipolar lines meet at epipoles. Hence in this case, as the epipoles are at infinity, our epipolar lines are parallel.

Awesome! This significantly simplifies the problem of dense point correspondence. However, we still have to perform triangulation for each point. Can we simplify this problem as well? Well, once again, the special case of parallel imaging planes has good news for us! It helps us to apply **stereo disparity**. It is similar to **stereopsis** or **stereoscopic vision**, the method that helps humans perceive depth. Let’s understand this in detail.

Understanding stereo disparity

The following gif is generated using images from the [Middlebury Stereo Datasets 2005 \(https://vision.middlebury.edu/stereo/data/scenes2005/\)](https://vision.middlebury.edu/stereo/data/scenes2005/). It demonstrates the pure translation motion of the camera, making the imaging planes parallel. Can you tell which objects are closer to the camera?



[Click here to download the source code to this post](#)

[./wp-content/uploads/2020/08/depth-from-motion-experiment.gif](#)

Gif showing left and right images where the imaging planes are parallel.

We can clearly say that the toy cow at the bottom is closer to the camera than the toys in the topmost row. How did we do this? We basically see the shift in the object in the two images. The more the shift closer is the object. This shift is what we call as **disparity**.

How do we use it to avoid point triangulation for calculating depth? We calculate the disparity (shift of the pixel in the two images) for each pixel and apply a proportional mapping to find the depth for a given disparity value. This is further justified in figure 12.

[./https://opencv-python-tutroals.readthedocs.io/en/latest/_images/stereo_depth.jpg](https://opencv-python-tutroals.readthedocs.io/en/latest/_images/stereo_depth.jpg)

Figure 12. Image from

[OpenCV documentation \(https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_calib3d/py_depthmap/py_depthmap.html\)](https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_calib3d/py_depthmap/py_depthmap.html)

explaining the relation between disparity ($x - x'$) and depth Z .

Disparity = $x - x' = Bf/Z$

Where B is the baseline (Distance between the cameras), and f is the focal length.

We will use a StereoSGBM method of OpenCV to write a code for calculating the disparity map for a given pair of images. The StereoSGBM method is based on [3].

Download Code To easily follow along this tutorial, please download code by clicking on the button below. It's FREE!

C++

```
1 // Reading the left and right images.
2
3 cv::Mat imgL, imgR;
4 imgL = cv::imread("../im0.png"); // path to left image is "../im0.png"
5 imgR = cv::imread("../im1.png"); // path to left image is "../im1.png"
6
7 // Setting parameters for StereoSGBM algorithm
8 int minDisparity = 0;
9 int numDisparities = 64;
10 int blockSize = 8;
11 int disp12MaxDiff = 1;
12 int uniquenessRatio = 10;
13 int speckleWindowSize = 10;
14 int speckleRange = 8;
15
16 // Creating an object of StereoSGBM algorithm
17 cv::Ptr<cv::StereoSGBM> stereo = cv::StereoSGBM::create(minDisparity, numDisparities, blockSize,
18 disp12MaxDiff, uniquenessRatio, speckleWindowSize, speckleRange);
19
20 // Calculating disparith using the StereoSGBM algorithm
21 cv::Mat disp;
22 stereo->compute(imgL, imgR, disp);
23
24 // Normalizing the disparity map for better visualisation
25 cv::normalize(disp, disp, 0, 255, cv::NORM_MINMAX, CV_8UC1);
26
27 // Displaying the disparity map
28 cv::imshow("disparity", disp);
29 cv::waitKey(0);
```

Python

```
1 # Reading the left and right images.
2
3 imgL = cv2.imread("../im0.png", 0)
4 imgR = cv2.imread("../im1.png", 0)
5
```



[Click here to download the source code to this post](#)

```
6 # Setting parameters for StereoSGBM algorithm
7 minDisparity = 0;
8 numDisparities = 64;
9 blockSize = 8;
10 disp12MaxDiff = 1;
11 uniquenessRatio = 10;
12 speckleWindowSize = 10;
13 speckleRange = 8;
14
15 # Creating an object of StereoSGBM algorithm
16 stereo = cv2.StereoSGBM_create(minDisparity = minDisparity,
17                               numDisparities = numDisparities,
18                               blockSize = blockSize,
19                               disp12MaxDiff = disp12MaxDiff,
20                               uniquenessRatio = uniquenessRatio,
21                               speckleWindowSize = speckleWindowSize,
22                               speckleRange = speckleRange
23                               )
24
25 # Calculating disparith using the StereoSGBM algorithm
26 disp = stereo.compute(imgL, imgR).astype(np.float32)
27 disp = cv2.normalize(disp,0,255,cv2.NORM_MINMAX)
28
29 # Displaying the disparity map
30 cv2.imshow("disparity",disp)
31 cv2.waitKey(0)
```

[Disprity-images-1024x339.jpg](#) [./wp-content/uploads/2020/10/Left-Right-and-](#)

Figure 13. Left, right images of a real world scenario and corresponding output disparity image of a real world scenario.
Source – 2014 High Resolution Stereo Datasets (<https://vision.middlebury.edu/stereo/data/scenes2014/>) [2]

Try playing with the different parameters to observe how they affect the final output disparity map calculation. A detailed explanation of the StereoSGBM will be presented in the subsequent **Introduction to Spatial AI series**. In the next post, we will learn to create our own stereo camera setup and record live disparity map videos, and we will also learn how to convert a disparity map into a depth map. An interesting application of stereo cameras will also be explained, but that is a surprise for now!

Subscribe & Download Code

If you liked this article and would like to download code (C++ and Python) and example images used in this post, please [click here](#). Alternately, sign up to receive a free [Computer Vision Resource](#) Guide. In our newsletter, we share OpenCV tutorials and examples written in C++/Python, and Computer Vision and Machine Learning algorithms and news.

References

[1] Richard Hartley and Andrew Zisserman. 2003. *Multiple View Geometry in Computer Vision* (2nd. ed.). Cambridge University Press, USA.

[2] D. Scharstein, H. Hirschmüller, Y. Kitajima, G. Krathwohl, N. Nesić, X. Wang, and P. Westling. High-resolution stereo datasets with subpixel-accurate ground truth. In German Conference on Pattern Recognition (GCPR 2014), Münster, Germany, September 2014.

[3] H. Hirschmuller, “Stereo Processing by Semiglobal Matching and Mutual Information,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328-341, Feb. 2008, doi: 10.1109/TPAMI.2007.1166.

Subscribe Now

About LearnOpenCV

Empowering innovation through education, LearnOpenCV provides in-depth tutorials, code, and guides in AI, Computer Vision, and Deep Learning. Led by Dr. Satya Mallick, we're dedicated to nurturing a community keen on technology breakthroughs.

[\(https://learnopencv.com/about/\)](https://learnopencv.com/about/)

Click here to download the source code to this post

- TensorFlow & Keras Bootcamp
- OpenCV Bootcamp
- Python for Beginners

- Deep Learning
- Object Detection
- Image Classification
- YOLO
- Image Processing
- Image Segmentation

Getting Started

- Installation
- PyTorch
- Getting Started with OpenCV
- Keras & Tensorflow

OpenCV University

- CVDL Master Program
- Student Discount
- CareerX

